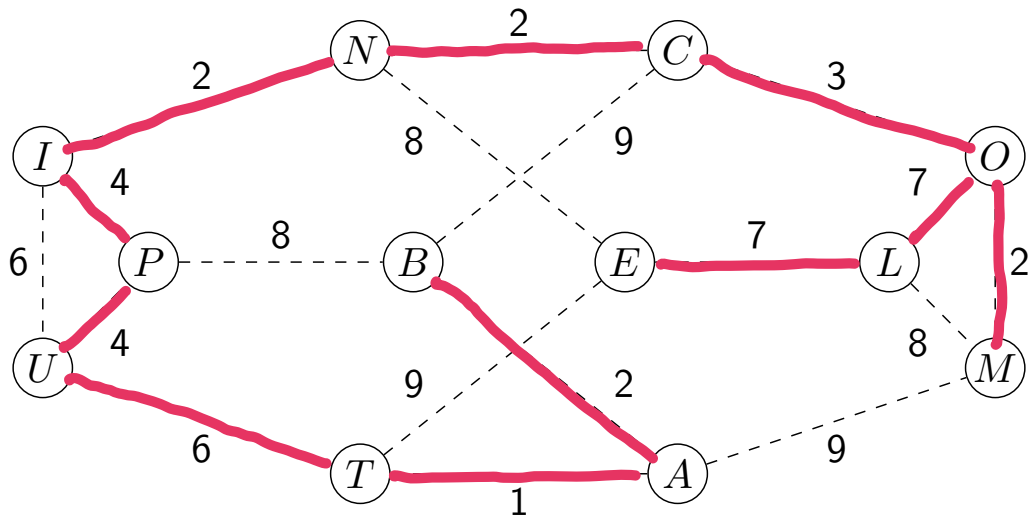
CSE2331

Graph Algorithms Homework

1 Minimum Spanning Tree

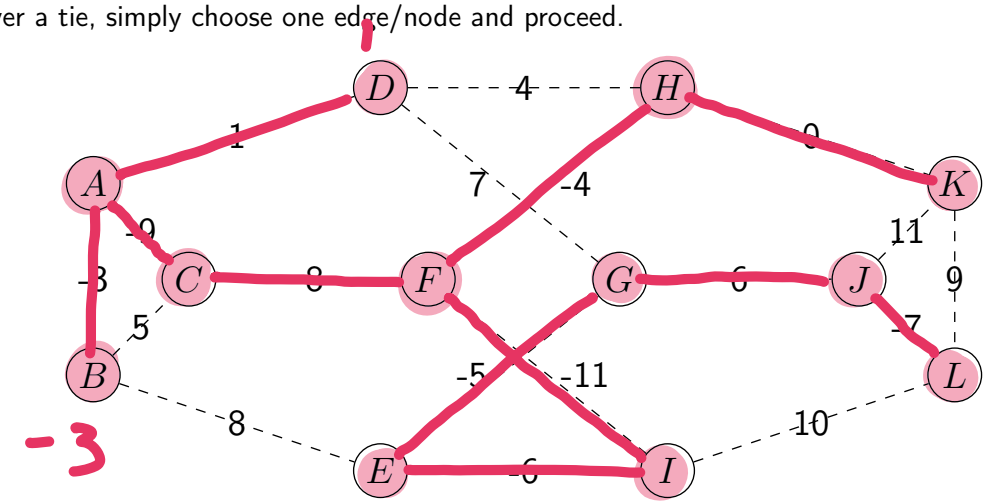
Problem 1.1. Use **Prim's Algorithm** starting at vertex I to determine the minimum spanning tree of the undirected weighted graph given below. List the order in which the vertices are added to the minimum spanning tree and draw the minimum spanning tree on the graph.
If there is ever a tie, simply choose one edge/node and proceed.



INCOMPUTABLE

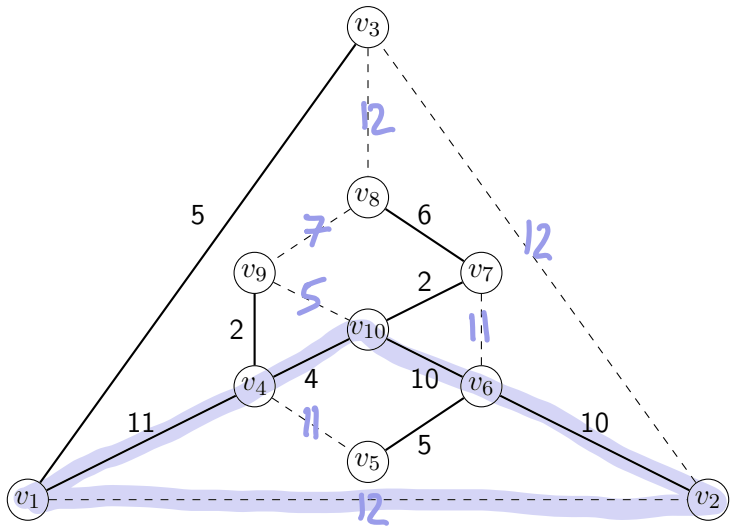
Problem 1.2. Use **Prim's Algorithm** starting at vertex A to determine the minimum spanning tree of the undirected weighted graph given below. List the order in which the vertices are added to the minimum spanning tree and draw the minimum spanning tree on the graph.

If there is ever a tie, simply choose one edge/node and proceed.



ACFIEGHBKDJL

Problem 1.3. Consider the graph given below,



The solid edges form a spanning tree T of graph G. Each of the solid edges has a weight. Assign weights to the dotted edges, (1,2), (2,3), (3,8), (4,5), (8,9), and (9,10), such that:

- Each of the edge weights is a positive INTEGER;
- Tree T is a MINIMUM spanning tree of G and NO other tree is a MINIMUM SPANNING TREE of G;
- Each of the edge weights of the dotted edges is as small as possible.

This problem is a type of inverse problem to the previous ones. In the previous problems we were given a graph and had to determine the minimum spanning tree. In this problem, we are given a minimum spanning tree and are trying to reverse engineer what type of graph could have given us such a tree.
Hint: The lemmas/theorems in the notes will help you solve this!

If edge (v_1, v_2) were added to the spanning tree, it would create the cycle $C = v_1, v_4, v_{10}, v_6, v_2, v_1$. By the MST Lemma, if $w(v_1, v_2) \leq w(e)$ for any edge e in the cycle, then we can add (v_1, v_2) and remove e from the tree and the total weight will not increase. Thus, $w(v_1, v_2) > w(e)$, $\forall e \in C$

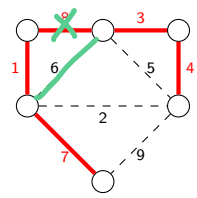
So, $w(v_1, v_{10}) = 12$.

All other edges are similar.

Problem 1.4. For each of the following weighted graphs a "potential" minimum spanning tree T is shown by highlighting the selected edges in **bold**. Some of these minimum spanning trees are valid and some are invalid. Identify whether T is a valid minimum spanning tree; explain why it is valid or invalid.

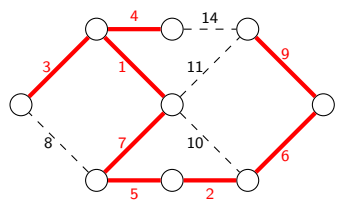
Hint: The theorems and lemmas from the notes will help you explain!

(a)



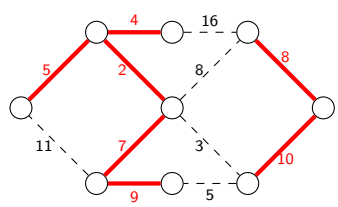
Invalid, not min.

(b)



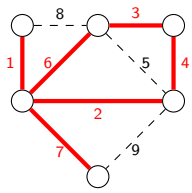
Valid, we obtain this tree when we run Kurskal's MST.

(c)



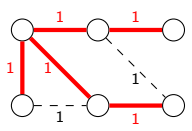
Invalid, not connected

(d)



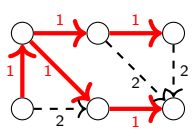
Invalid, cycle!

(e)



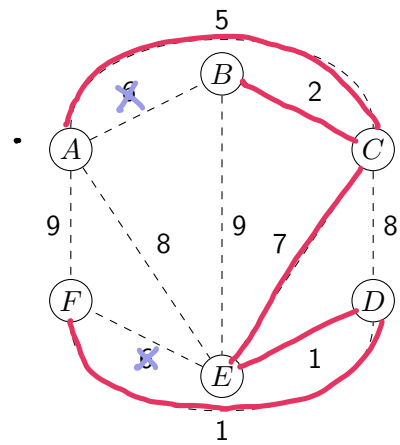
Valid, all spanning trees have same weight.

(f)



Invalid, not strongly connected

Problem 1.5. Use **Kruskal's Algorithm** to find a minimum spanning tree of the undirected weighted graph given below. At each step of the algorithm list the current disjoint sets that are being stored.
If there is ever a tie, simply choose one edge/node and proceed.



Union(D,E)

{DE} {A} {B} {C} {F}

Union(D,F)

{DEF} {A} {B} {C}

Union(B,C)

{DEF} {A} {BC}

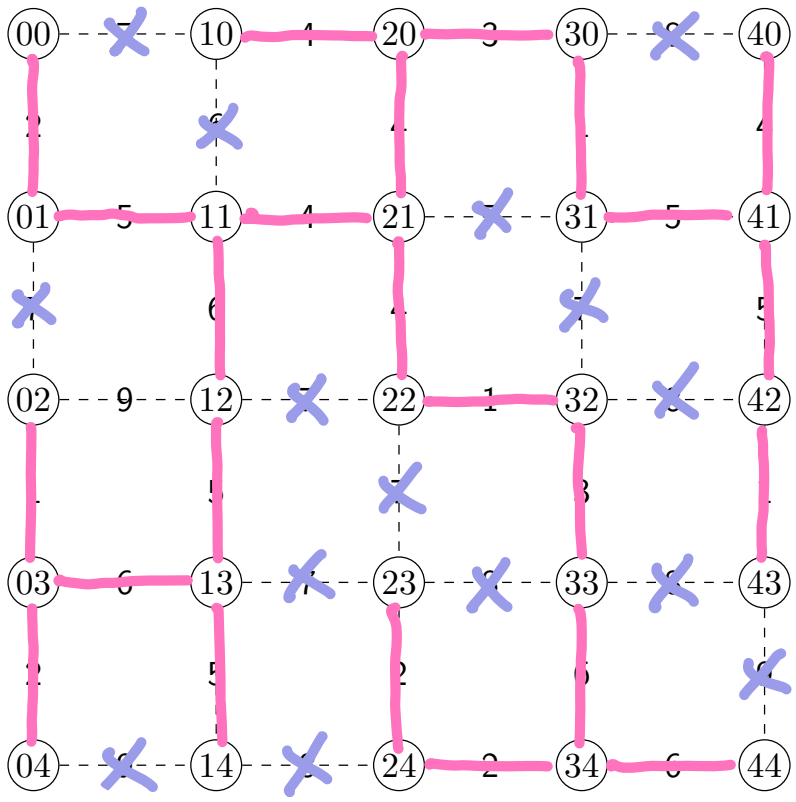
Union(A,C)

{DEF} {ABC}

Union(C,E)

{DEFABC}

Problem 1.6. Kruskal's algorithm is commonly used for Maze Generation. A maze without any cycles is equivalent to a tree in a graph. Consider the following graph,

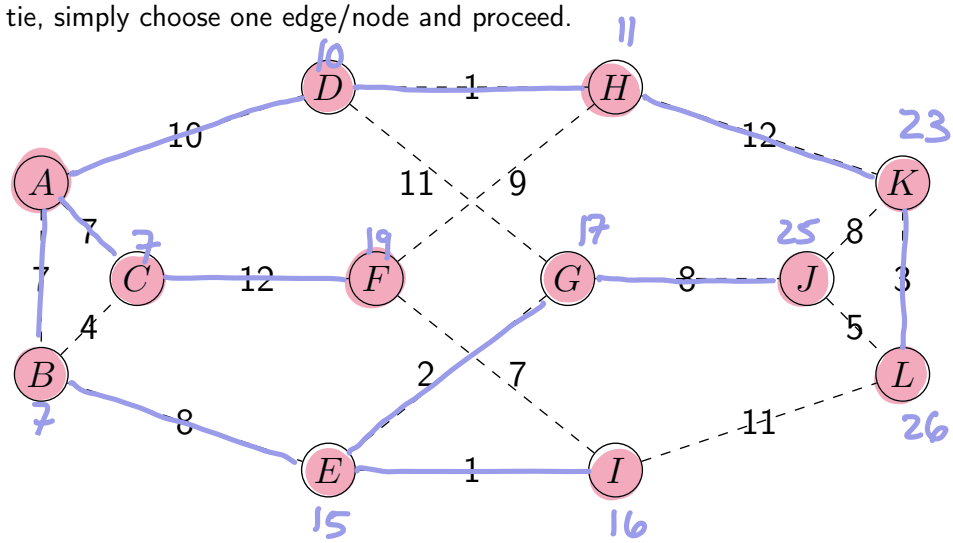


Apply Kruskal's to find the minimum spanning tree for this graph. **You do not need to show your work.**

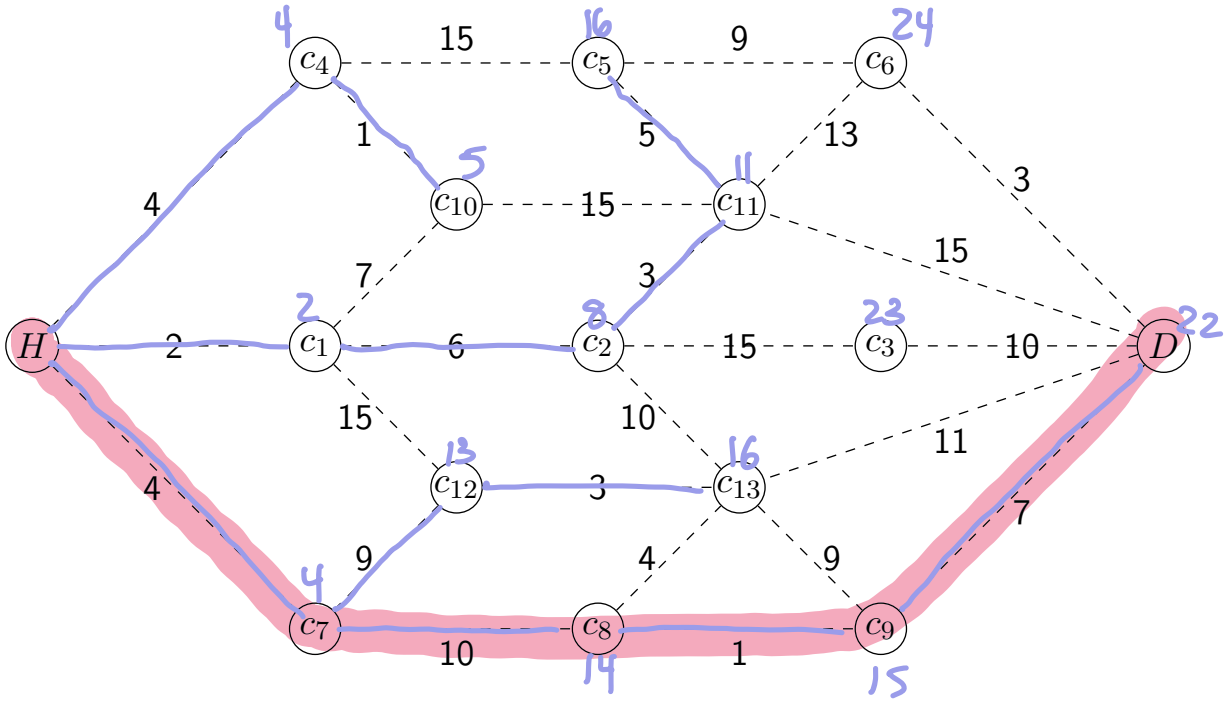
2 Shortest Paths

Problem 2.1. Use **Dijkstra's Algorithm** starting at vertex A to determine the shortest path from A to every other node in the tree. List the order in which the vertices are added to the shortest path tree and draw the spanning tree generated by the algorithm. In addition to determining the shortest path, indicate the distance from A to every other vertex.

If there is ever a tie, simply choose one edge/node and proceed.



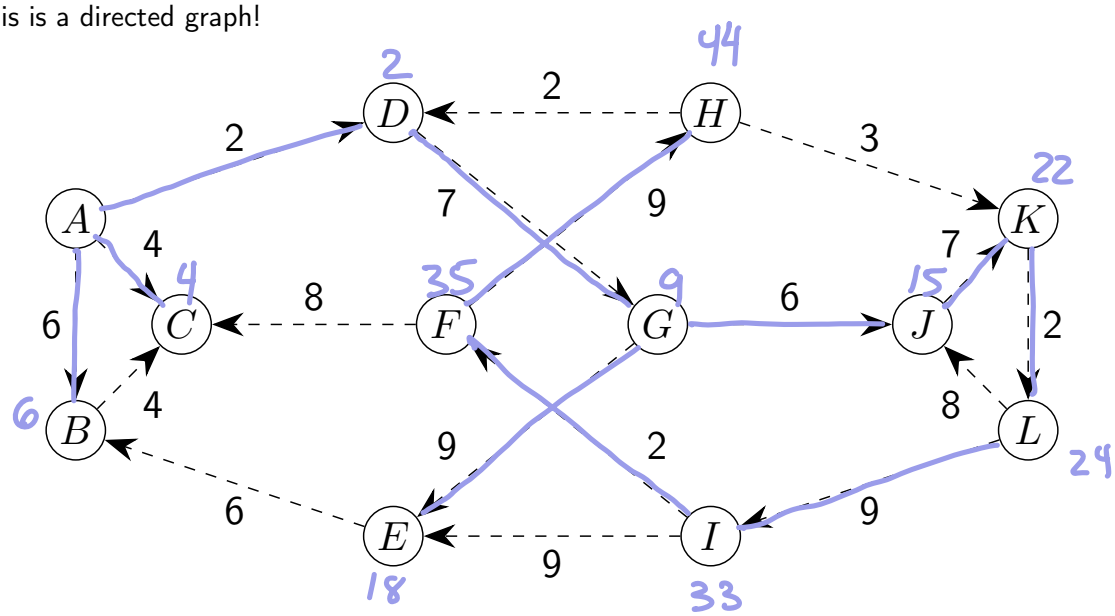
Problem 2.2. Consider the following graph. This graph corresponds to a set of intersections in Columbus – each intersection is a node in the graph and each edge is a street that leads between the intersections. Your goal is use Djikstra's Algorithm to find the fastest route from vertex H (your home address) to vertex D for Dreese Labs. You must perform Djikstra's as describe in the notes – that is, be sure to always visit the next closest node even if that node does not get used in the final shortest path from H to D.



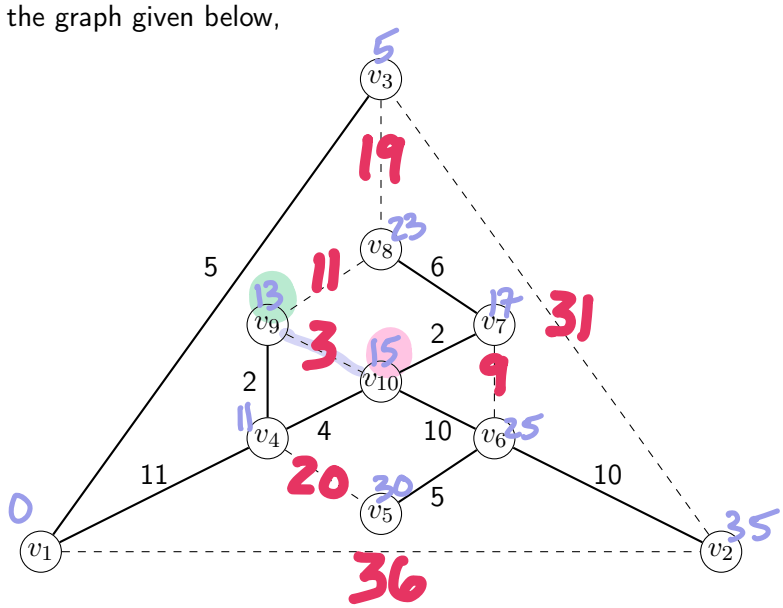
Problem 2.3. Use **Dijkstra's Algorithm** starting at vertex A to determine the shortest path from A to every other node in the tree. List the order in which the vertices are added to the shortest path tree and draw the spanning tree generated by the algorithm. In addition to determining the shortest path, indicate the distance from A to every other vertex.

If there is ever a tie, simply choose one edge/node and proceed.

Note: This is a directed graph!



Problem 2.4. Consider the graph given below,



The solid edges form a spanning tree T of graph G. Each of the solid edges has a weight. Assign weights to the dotted edges, (1,2), (2,3), (3,8), (4,5), (8,9), and (9,10), such that:

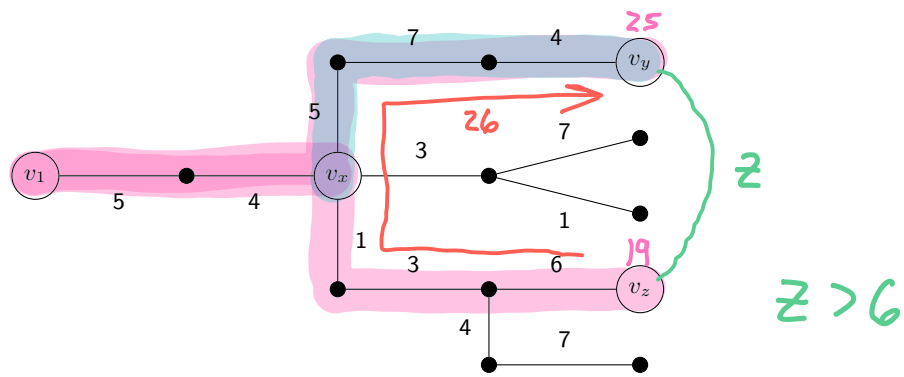
- Each of the edge weights is a positive INTEGER;
- Tree T is a SHORTEST PATH tree of G with root v_1 and NO other tree is a SHORTEST PATH TREE of G;
- Each of the edge weights of the dotted edges is as small as possible.

If we consider the edge (v_9, v_{10}) we must ensure that $d(v_1, v_9) + w(v_9, v_{10}) > d(v_1, v_{10})$ to ensure that $v_1 \to v_{10} \to v_9$ is not a "short cut". So,

$$\left.\begin{aligned} 13 + w(v_{10}, v_9) &> 15 \\ w(v_{10}, v_9) &> 2 \end{aligned}\right\} \text{set } w(v_{10}, v_9) = 3.$$

All other edges similar.

Problem 2.5. Let G be a positive edge weighted graph whose **SHORTEST PATH TREE** T from vertex v_1 is:



Note that the above graph is the shortest path tree, NOT the graph G. Graph G has other edges and weights on those edges, but this information is NOT available.

(a) Using only this information, can you determine the shortest path from v_x to v_y? If yes, give the length of the path. If not, give as accurate as possible upper and lower bounds on the length of the path.

(b) Using only this information, can you determine the shortest path from v_y to v_z? If yes, give the length of the path. If not, give as accurate as possible upper and lower bounds on the length of the path.

Explain your reasoning.

a) Yes! Since $v_x \to v_y$ is a subpath of the path $v_1 \to v_x \to v_y$. $L = 16$

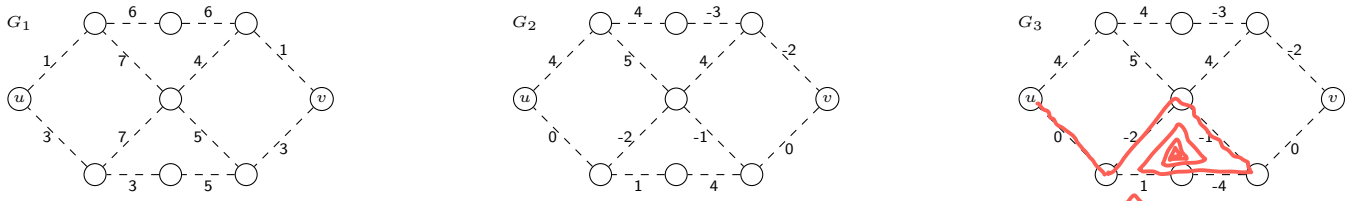
Subpaths of shortest paths are shortest paths.

b) No! In the picture if $z = 15$ then the shortest path is length 15, which cannot be discovered from the tree. In general,

$z > 6$ because otherwise $v_1 \to \cdots \to v_z \to v_y$ would be a shorter path from v_1 to v_y.

The shortest path is also ≤ 26 since there is a path with length 26.

Problem 2.6. Consider the following graphs,



One of these graphs has **no shortest path from** u **to** v. Which graph has no shortest path from u to v and **why** is there no shortest path?

Note that shortest path can repeat vertices.

you can circle endlessly and since weight of cycle < 0, it means any shortest path can be made shorter by cycling more.